Universidad Mariano Gálvez de Guatemala
Ingeniería en Sistemas
Curso Desarrollo Web



Sistema de Gestión de Biblioteca
Proyecto final

0900-23-17900	Ximena Aguilar
0900-23-16244	Joselin Vasquez
0900-23-10488	Flor de María Ponce
0900-23-7115	José Mata

Ing. Aaron Velasquez
Sección A
Fecha: 31/10/2026

Índice

Índice 2
Introducción 4
Problema y justificación 5
Objetivos 5
Objetivo general 5
Objetivos específicos 5
Alcance y organización 5
Requisitos funcionales 6
Reglas de negocio y calidad 6
Reglas de negocio 6
Validación de datos 7
Seguridad básica 7
Control de préstamos simultáneos 8
Arquitectura 8
2 API REST 9
3 Base de datos 9
4 Comunicación entre los componentes 9
Diseño de la base de datos 10
1 Relaciones entre las tablas 10
3 Integridad y restricciones 11
4 Determinación de la disponibilidad 11
5 Datos iniciales 12
Herramientas y entorno 12
Plantilla de implementación y resultados 13
1 Implementación de la API 13
2 Endpoints implementados 13
3 Conexión con MySQL 14
Implementación de la interfaz 15
1 Vista de libros 15
2 Vista de usuarios 16
3 Vista de préstamos 17
4 Eventos y comunicación con la API 17

5 Validaciones y tratamiento de errores 18
Instalación y ejecución 18
1 Obtención del código 18
2 Instalación de dependencias 18
3 Creación de la base de datos 18
4 Configuración del usuario de aplicación 19
5 Variables de entorno 19
6 Ejecución 19
7 Verificación de instalación limpia 20
Pruebas y evidencias 20
2 Evidencia de persistencia 24
Dificultades y aprendizajes 25
1 Dificultades encontradas 25
2 Aprendizajes obtenidos 25
Conclusiones y mejoras 26
1 Conclusiones 26
2 Mejoras propuestas 26
Referencias y anexos 27

Introducción

El presente proyecto consiste en el desarrollo de un Sistema de Gestión de Biblioteca que permite administrar ejemplares, registrar usuarios y controlar préstamos y devoluciones mediante una aplicación web.

La solución integra una interfaz construida con HTML, CSS y JavaScript, una API REST desarrollada con Node.js y Express, y una base de datos MySQL. La comunicación entre la interfaz y el servidor se realiza mediante solicitudes HTTP utilizando fetch() y async/await.

El sistema permite consultar la disponibilidad de los ejemplares y conservar el historial de préstamos. Además, incorpora validaciones para impedir operaciones incorrectas, como prestar un ejemplar que todavía no ha sido devuelto.

Este informe presenta el problema planteado, los objetivos, la arquitectura, el diseño de la base de datos, las funcionalidades implementadas y las evidencias de las pruebas realizadas.

Problema y justificación

El escenario planteado requiere controlar los libros de una biblioteca, identificar qué ejemplares están disponibles y conocer qué usuarios tienen préstamos pendientes. Cuando estos registros se mantienen de forma desorganizada, pueden presentarse dificultades para consultar la información y dar seguimiento a las devoluciones.

Por ello, se desarrolló una aplicación que centraliza los registros de libros, usuarios y préstamos en una base de datos relacionada. La solución permite consultar información actualizada y aplicar la regla de negocio que impide prestar un ejemplar ocupado.

El proyecto también permite demostrar la integración de las capas de una aplicación Full Stack, desde una acción del usuario en el navegador hasta la consulta o modificación de información persistente en MySQL.

Objetivos

Objetivo general

Desarrollar una aplicación web Full Stack para administrar los ejemplares, usuarios y préstamos de una biblioteca, mediante una interfaz conectada a una API REST y una base de datos MySQL.

Objetivos específicos

- Implementar el registro, consulta, actualización y eliminación de ejemplares.
- Permitir el registro y la consulta de usuarios de la biblioteca.
- Controlar los préstamos y las devoluciones, conservando su historial.
- Identificar la disponibilidad de cada ejemplar.
- Validar los datos ingresados e impedir préstamos duplicados de un mismo ejemplar.
- Comprobar el funcionamiento de la API, la interfaz y la persistencia mediante pruebas.

Alcance y organización

El sistema permite administrar ejemplares físicos de libros, registrar lectores y gestionar préstamos y devoluciones. Cada ejemplar cuenta con un código único, por lo que dos copias del mismo título pueden administrarse de forma independiente.

La interfaz incluye las secciones Libros, Usuarios y Préstamos. Desde ellas es posible consultar información, registrar datos y ejecutar las operaciones implementadas por la API.

La disponibilidad se determina a partir de los préstamos pendientes de devolución. Los ejemplares con historial no pueden eliminarse, para conservar la relación entre los libros, los usuarios y los préstamos realizados.

La aplicación se ejecuta localmente. El alcance desarrollado no incluye autenticación, roles de acceso, reservas ni multas.

Requisitos funcionales

ID	Requisito	Implementación
RF01	Registrar libros	Permite ingresar código, título, autor e ISBN opcional.
RF02	Consultar libros	Muestra el catálogo, permite buscar y consultar un ejemplar por su ID mediante la API.
RF03	Actualizar libros	Permite modificar los datos de un ejemplar registrado.
RF04	Eliminar libros	Permite eliminar ejemplares que no tienen historial de préstamos.
RF05	Registrar usuarios	Permite ingresar nombre, correo electrónico y teléfono opcional.
RF06	Consultar usuarios	Muestra los lectores registrados en la biblioteca.
RF07	Registrar préstamos	Relaciona un ejemplar disponible con un usuario y una fecha límite.
RF08	Consultar historial	Muestra el ejemplar, usuario, fechas y estado de cada préstamo.
RF09	Registrar devoluciones	Registra la fecha de devolución y libera el ejemplar.
RF10	Identificar disponibilidad	Determina si un ejemplar tiene algún préstamo pendiente de devolución.
RF11	Validar información	Rechaza campos inválidos, registros duplicados y operaciones incorrectas.

Reglas de negocio y calidad

Reglas de negocio

ID	Regla
RN01	Cada ejemplar físico tiene un código único. Dos copias del mismo título se registran con códigos diferentes.
RN02	No se permite registrar un préstamo cuando el ejemplar tiene otro pendiente de devolución.
RN03	Un préstamo está pendiente de devolución cuando fecha_devolucion es NULL.
RN04	Un préstamo vencido mantiene el ejemplar ocupado hasta registrar su devolución.
RN05	La fecha límite no puede ser anterior al día en que se registra el préstamo.
RN06	Solo se pueden registrar préstamos con libros y usuarios existentes.
RN07	Un préstamo devuelto no puede registrar una segunda devolución.
RN08	Los ejemplares con historial de préstamos no pueden eliminarse.
RN09	Cada usuario debe tener un correo electrónico único.
RN10	Registrar una devolución conserva el préstamo en el historial.

Validación de datos

Dato u operación	Validación
Código del ejemplar	Obligatorio, único y de hasta 30 caracteres.
Título	Obligatorio y de hasta 200 caracteres.
Autor	Obligatorio y de hasta 150 caracteres.
ISBN	Opcional, de tipo texto y de hasta 20 caracteres.
Nombre del usuario	Obligatorio y de hasta 150 caracteres.
Correo	Formato básico de correo válido, único y de hasta 254 caracteres.
Teléfono	Opcional; cuando se proporciona, debe cumplir el formato permitido de 7 a 25 caracteres.
Identificadores	Deben ser números enteros positivos.
Fecha límite	Debe ser una fecha real y no anterior al día del préstamo.
Solicitudes JSON	Deben tener una estructura válida y respetar el límite de 10 KB.

El ISBN actualmente se valida por tipo y longitud; no implementamos una comprobación del dígito verificador de ISBN.

Seguridad básica

Medida	Aplicación en el proyecto
Consultas parametrizadas	Los valores ingresados se envían separados de la instrucción SQL mediante marcadores ?.
Variables de entorno	La configuración de conexión se obtiene del archivo .env.
Exclusión de credenciales	.gitignore excluye .env del repositorio.
Permisos limitados	biblioteca_app tiene permisos SELECT, INSERT, UPDATE y DELETE sobre la base utilizada.
Presentación de datos	La interfaz utiliza textContent para mostrar información ingresada por los usuarios.
Manejo de errores	La API devuelve mensajes controlados sin enviar contraseñas ni detalles SQL al navegador.
Integridad referencial	Las claves foráneas impiden relaciones inválidas y la eliminación de ejemplares con préstamos.

Control de préstamos simultáneos

El sistema incorpora un mecanismo para controlar las solicitudes que intentan prestar un mismo ejemplar al mismo tiempo. Sin este control, dos solicitudes podrían consultar que un libro está disponible y registrar dos préstamos antes de detectar que el ejemplar ya fue asignado.

Para evitar esta situación, el registro de préstamos se realiza mediante una transacción en MySQL. Todas las operaciones de la transacción utilizan una misma conexión con la base de datos.

Primero, el sistema consulta y bloquea temporalmente el registro del ejemplar mediante la instrucción SELECT ... FOR UPDATE. Si otra solicitud intenta bloquear ese mismo ejemplar, debe esperar a que finalice la primera transacción.

Después, el sistema comprueba que el usuario exista, valida la fecha límite y consulta si el ejemplar tiene algún préstamo pendiente de devolución. Esta última consulta también utiliza FOR UPDATE para comprobar el estado actual de los registros.

Si el ejemplar está disponible, el sistema registra el préstamo y confirma los cambios mediante COMMIT. Si ya está prestado, revierte la transacción mediante ROLLBACK y devuelve el código HTTP 409, acompañado de un mensaje que explica el conflicto.

Cuando ocurre un error, también se intenta revertir la transacción. Finalmente, la conexión se libera para que pueda ser utilizada por otras solicitudes.

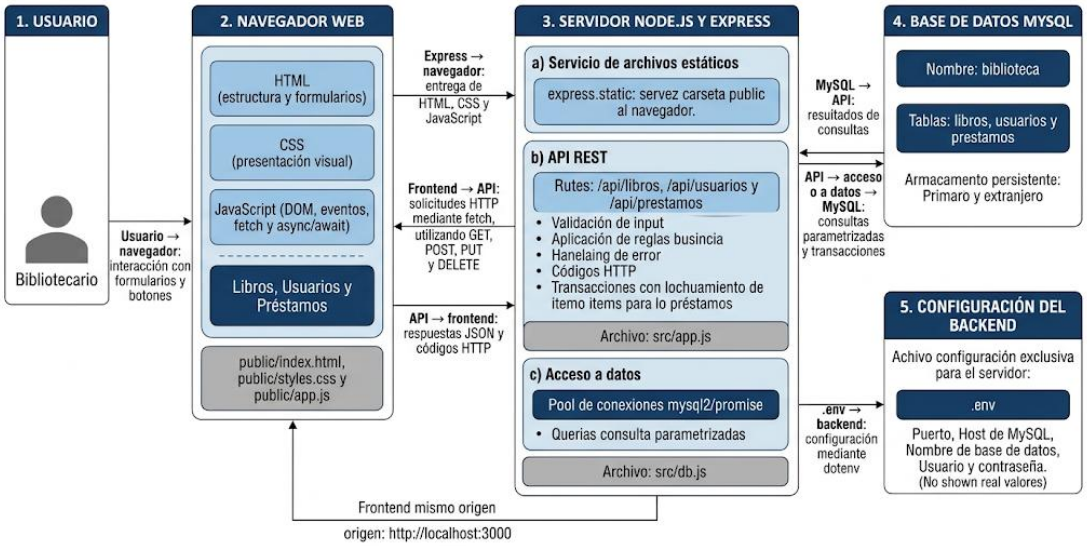
Este mecanismo está implementado en la ruta POST /api/prestamos. La prueba realizada mediante solicitudes consecutivas comprobó el rechazo de un segundo préstamo activo. La verificación mediante solicitudes simultáneas se mantiene pendiente de ejecución.

Arquitectura

El Sistema de Gestión de Biblioteca utiliza una arquitectura que integra una interfaz web, una API REST y una base de datos relacional. Cada componente tiene una responsabilidad definida y se comunica con los demás para completar las operaciones solicitadas por el usuario.

La interfaz permite introducir y consultar información. La API valida las solicitudes, aplica las reglas de negocio y ejecuta las operaciones correspondientes. MySQL almacena los datos de forma persistente y mantiene las relaciones entre libros, usuarios y préstamos.

Arquitectura del Sistema de Gestión de Biblioteca



1 Interfaz web

La interfaz se desarrolló con HTML, CSS y JavaScript. HTML define la estructura de las secciones y formularios, CSS organiza la presentación visual y JavaScript administra los eventos y la actualización del contenido.

La aplicación incluye las secciones Libros, Usuarios y Préstamos. Desde ellas se pueden registrar datos, consultar información y ejecutar las operaciones disponibles.

El archivo public/app.js utiliza fetch() para comunicarse con la API. Las solicitudes se procesan mediante async/await, y los errores se presentan al usuario a través de mensajes en la página.

2 API REST

La API se desarrolló con Node.js y Express. Sus rutas reciben solicitudes HTTP, validan los datos y determinan qué operación debe ejecutarse en MySQL.

Se utilizan los métodos GET para consultar, POST para registrar, PUT para actualizar y DELETE para eliminar los registros permitidos.

La API intercambia información en formato JSON y devuelve códigos HTTP que permiten identificar si una operación fue exitosa, si los datos son inválidos o si existe algún conflicto con las reglas del sistema.

3 Base de datos

MySQL almacena la información en las tablas libros, usuarios y prestamos. Las claves primarias identifican cada registro y las claves foráneas relacionan los préstamos con sus respectivos ejemplares y usuarios.

El backend se conecta mediante la biblioteca mysql2, utilizando un pool de conexiones. Los parámetros de conexión se obtienen de variables de entorno.

La aplicación utiliza el usuario biblioteca_app, con permisos de consulta, inserción, actualización y eliminación sobre la base de datos correspondiente.

4 Comunicación entre los componentes

Cuando el usuario registra un préstamo desde la interfaz, JavaScript captura los datos del formulario y envía una solicitud POST a la ruta /api/prestamos.

Express recibe los datos en formato JSON y valida los identificadores y la fecha límite. Después, el backend obtiene una conexión MySQL e inicia la transacción que comprueba la disponibilidad del ejemplar.

Si la operación es válida, MySQL almacena el préstamo y la API responde con el código HTTP 201. La interfaz consulta nuevamente la información y actualiza el historial y la disponibilidad de los libros.

Si el ejemplar ya está prestado, la API responde con el código HTTP 409 y la interfaz muestra el mensaje correspondiente.

Los archivos de la interfaz se sirven desde Express mediante express.static. Por ello, la página y la API se utilizan desde el mismo origen, disponible localmente en http://localhost:3000.

Diseño de la base de datos

La base de datos se denomina biblioteca y está formada por tres tablas relacionadas: libros, usuarios y prestamos.

El diseño representa cada ejemplar físico como un registro independiente. Por ejemplo, dos copias de El principito pueden compartir título y autor, pero deben tener códigos de ejemplar diferentes. Esto permite controlar su disponibilidad y sus préstamos por separado.

Modelo entidad relación de la base de datos biblioteca



Notes:

- Cada row in libros representa una copia física.
- Dos copias de ms mismo título tienen tene dos códigos distintos.
- ISBN is opcional y no único.
- fecha_devolucion = NULL indicate un prestamo pendiente.
- Disponibilidad is calculada desde los prestamos pendientes y no es almaenada como un campo disponibilidad en libros.
- Solo de un prestamo pendiente por copia, cloiquiddo enforceo en la API via a la transacción de transacción y UNIQUE encanta FOR UPDATE lock porque constrainte on libro_id.

1 Relaciones entre las tablas

La tabla libros mantiene una relación de uno a muchos con prestamos. Un ejemplar puede participar en varios préstamos a lo largo del tiempo, aunque el sistema solo permite uno pendiente de devolución a la vez.

La tabla usuarios mantiene una relación de uno a muchos con prestamos. Un usuario puede tener varios préstamos registrados, y cada préstamo pertenece a un único usuario.

La tabla prestamos contiene las claves foráneas libro_id y usuario_id, que permiten identificar el ejemplar y el lector relacionados con cada operación.

2 Diccionario de datos

Las siguientes tablas describen los campos utilizados en la base de datos.

Campo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del ejemplar.
codigo	VARCHAR(30)	NOT NULL, UNIQUE	Código único del ejemplar físico.
titulo	VARCHAR(200)	NOT NULL	Título de la obra.

autor	VARCHAR(150)	NOT NULL	Autor de la obra.
isbn	VARCHAR(20)	Permite NULL	ISBN opcional; puede repetirse entre ejemplares.

Campo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del lector.
nombre	VARCHAR(150)	NOT NULL	Nombre del usuario.
correo	VARCHAR(254)	NOT NULL, UNIQUE	Correo electrónico del usuario.
telefono	VARCHAR(25)	Permite NULL	Teléfono opcional de contacto.

Campo	Tipo	Restricciones	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador del préstamo.
libro_id	INT	NOT NULL, FK	Referencia a libros.id.
usuario_id	INT	NOT NULL, FK	Referencia a usuarios.id.
fecha_prestamo	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Fecha y hora de registro del préstamo.
fecha_limite	DATE	NOT NULL	Fecha prevista para devolver el ejemplar.
fecha_devolucion	DATETIME	Permite NULL	Fecha y hora de devolución; NULL indica devolución pendiente.

3 Integridad y restricciones

Las tablas utilizan el motor InnoDB, que permite trabajar con transacciones y claves foráneas. La base se creó con el conjunto de caracteres utf8mb4, que permite almacenar caracteres especiales y letras acentuadas.

Los códigos de ejemplar y los correos electrónicos tienen restricciones UNIQUE para impedir registros duplicados. Los campos obligatorios utilizan NOT NULL y los identificadores principales se generan mediante AUTO_INCREMENT.

Las claves foráneas de la tabla prestamos utilizan ON DELETE RESTRICT. Esta restricción impide eliminar un libro o usuario mientras existan préstamos asociados, conservando la integridad de las relaciones y el historial.

La tabla prestamos incluye el índice compuesto idx_prestamo_activo, formado por libro_id y fecha_devolucion, para apoyar las consultas que identifican préstamos pendientes de un ejemplar.

4 Determinación de la disponibilidad

La disponibilidad no se almacena como un campo independiente en la tabla libros. Se calcula consultando si existe un préstamo del ejemplar cuya fecha de devolución sea NULL.

Cuando existe un préstamo pendiente, el ejemplar aparece como Prestado. Cuando no existe ninguno, aparece como Disponible.

Este procedimiento evita mantener dos registros separados del estado que podrían quedar desactualizados entre sí.

5 Datos iniciales

El archivo database.sql contiene la estructura de la base y los datos iniciales para la demostración: seis ejemplares, tres usuarios y dos préstamos.

Uno de los préstamos iniciales está pendiente de devolución y el otro ya fue devuelto. También se incluyen dos ejemplares de El principito con códigos diferentes, para demostrar la administración independiente de copias de un mismo título.

Durante las pruebas se registraron y modificaron datos adicionales. Por ello, las cantidades de la base utilizada durante el desarrollo pueden diferir de los valores iniciales del script.

La instalación del script se comprobó en una base independiente denominada biblioteca_verificacion, donde se verificó la creación de las tres tablas y la carga de los registros iniciales.

Herramientas y entorno

El proyecto se desarrolló utilizando herramientas para construir la interfaz, implementar la API, administrar la base de datos y comprobar la comunicación entre los componentes.

Visual Studio Code se utilizó para editar los archivos del proyecto y ejecutar los comandos de Node.js, npm y Git. MySQL Workbench permitió crear las tablas, insertar datos iniciales y consultar la información almacenada.

Postman se utilizó para probar los endpoints y verificar los cuerpos de respuesta y códigos HTTP. Git permitió registrar los avances durante el desarrollo, mientras que GitHub se utilizó para almacenar y compartir el código fuente.

Herramienta o tecnología	Uso en el proyecto
HTML5	Estructura de las páginas y formularios.
CSS3	Presentación visual y adaptación a diferentes tamaños de pantalla.
JavaScript	Eventos, manipulación del DOM y consumo de la API.
Node.js	Ejecución del backend.
Express	Rutas HTTP, archivos estáticos y manejo de solicitudes.
MySQL Server	Almacenamiento persistente de la información.
mysql2/promise	Consultas y administración de conexiones a MySQL.
dotenv	Lectura de variables de entorno.
Visual Studio Code	Edición del código y uso de la terminal.
MySQL Workbench	Administración y consultas de la base de datos.
Postman	Pruebas y colección de solicitudes de la API.

Git y GitHub	Control de versiones y repositorio del proyecto.

La aplicación se ejecuta localmente mediante Node.js y se consulta desde el navegador en la dirección http://localhost:3000.

MySQL almacena la información en la base biblioteca. El backend utiliza el usuario biblioteca_app, que cuenta con permisos SELECT, INSERT, UPDATE y DELETE sobre esa base.

Los parámetros de conexión se configuran mediante el archivo .env. Este archivo permanece fuera del repositorio para evitar la publicación de credenciales. El archivo .env.example presenta las variables necesarias con valores de ejemplo.

Las dependencias del proyecto se registran en package.json, y sus versiones resueltas se conservan en package-lock.json.

Componente	Versión utilizada
Node.js	v24.21.0
npm	11.19.0
MySQL Server	26.10.0

Plantilla de implementación y resultados

1 Implementación de la API

La API REST permite consultar y modificar los datos de la biblioteca mediante solicitudes HTTP. Se desarrolló con Node.js y Express, y sus rutas se encuentran en el archivo src/app.js.

El backend recibe información en formato JSON, valida los datos y ejecuta consultas parametrizadas mediante mysql2/promise. Las respuestas incluyen códigos HTTP que permiten distinguir las operaciones exitosas de los errores o conflictos.

2 Endpoints implementados

Los endpoints se organizan alrededor de los recursos libros, usuarios y préstamos. También se incluye una ruta de diagnóstico para comprobar la conexión con MySQL.

Método	Endpoint	Función	Respuesta exitosa
GET	/api/health	Comprobar conexión con MySQL.	200
GET	/api/libros	Consultar ejemplares y disponibilidad.	200
GET	/api/libros/:id	Consultar un ejemplar específico.	200
POST	/api/libros	Registrar un ejemplar.	201
PUT	/api/libros/:id	Actualizar sus datos.	200
DELETE	/api/libros/:id	Eliminar un ejemplar sin historial.	200
GET	/api/usuarios	Consultar usuarios.	200

POST	/api/usuarios	Registrar un usuario.	201
GET	/api/prestamos	Consultar el historial de préstamos.	200
POST	/api/prestamos	Registrar un préstamo.	201
PUT	/api/prestamos/:id/devolucion	Registrar la devolución de un préstamo.	200

3 Conexión con MySQL

El archivo src/db.js crea un pool de conexiones utilizando mysql2/promise. Este mecanismo administra las conexiones que necesita la API para ejecutar consultas.

La configuración utiliza variables de entorno para definir el host, puerto, usuario, contraseña y nombre de la base de datos. El archivo .env se carga mediante dotenv antes de importar el módulo de conexión.

Las operaciones comunes utilizan el pool directamente. Para registrar préstamos, se obtiene una conexión específica, ya que todas las instrucciones de la transacción deben ejecutarse en la misma sesión de MySQL.

4 Consultas parametrizadas

Las operaciones que reciben valores del usuario utilizan marcadores ? en las instrucciones SQL. Los valores se proporcionan mediante un arreglo separado, evitando construir las consultas mediante concatenación de entradas.

5 Parámetros de ruta y cuerpo de solicitud

Los parámetros de ruta permiten identificar el recurso sobre el que se realiza una operación. Por ejemplo, en /api/libros/3, el valor 3 se obtiene mediante req.params.id.

El cuerpo de la solicitud contiene los datos enviados para registrar o actualizar información. Express procesa el JSON mediante express.json(), y el backend accede a sus campos a través de req.body.

En una actualización de libro, el identificador se recibe en la URL y los nuevos valores de código, título, autor e ISBN se reciben en el cuerpo de la solicitud.

6 Manejo de errores

La API valida los datos antes de ejecutar las operaciones y controla los errores que pueden producirse durante el acceso a MySQL.

Cuando no existe un recurso, responde con HTTP 404. Los datos inválidos generan HTTP 400, mientras que los conflictos, como códigos duplicados o ejemplares ocupados, generan HTTP 409.

Los errores inesperados se responden con HTTP 500 y un mensaje general. La ruta /api/health devuelve HTTP 503 cuando no puede conectarse con MySQL.

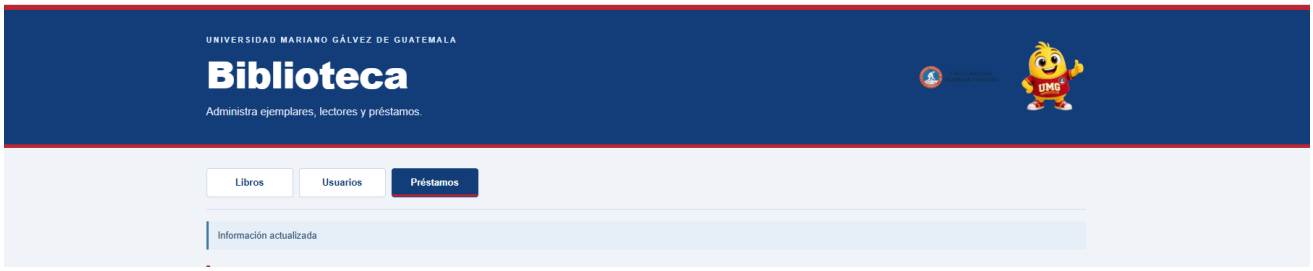
Los detalles técnicos necesarios para revisar errores se registran en la consola del servidor, sin enviar contraseñas ni instrucciones SQL al navegador.

Código HTTP	Uso
200 OK	Consulta, actualización, eliminación o devolución completada.
201 Created	Libro, usuario o préstamo registrado.
400 Bad Request	Datos inválidos o JSON incorrecto.
404 Not Found	Recurso o ruta inexistente.
409 Conflict	Duplicado o conflicto con una regla de negocio.
413 Content Too Large	Cuerpo JSON superior al límite permitido.
500 Internal Server Error	Error inesperado durante una operación.
503 Service Unavailable	Conexión MySQL no disponible en la ruta de diagnóstico.

Implementación de la interfaz

La interfaz del Sistema de Gestión de Biblioteca se desarrolló con HTML, CSS y JavaScript. Se organiza en tres secciones: Libros, Usuarios y Préstamos. Estas permiten realizar las operaciones del sistema desde formularios y botones conectados con la API REST.

HTML define la estructura de los formularios y tablas; CSS establece la presentación visual y adapta la distribución a diferentes tamaños de pantalla. JavaScript administra los eventos, envía solicitudes al backend y actualiza la información mostrada.

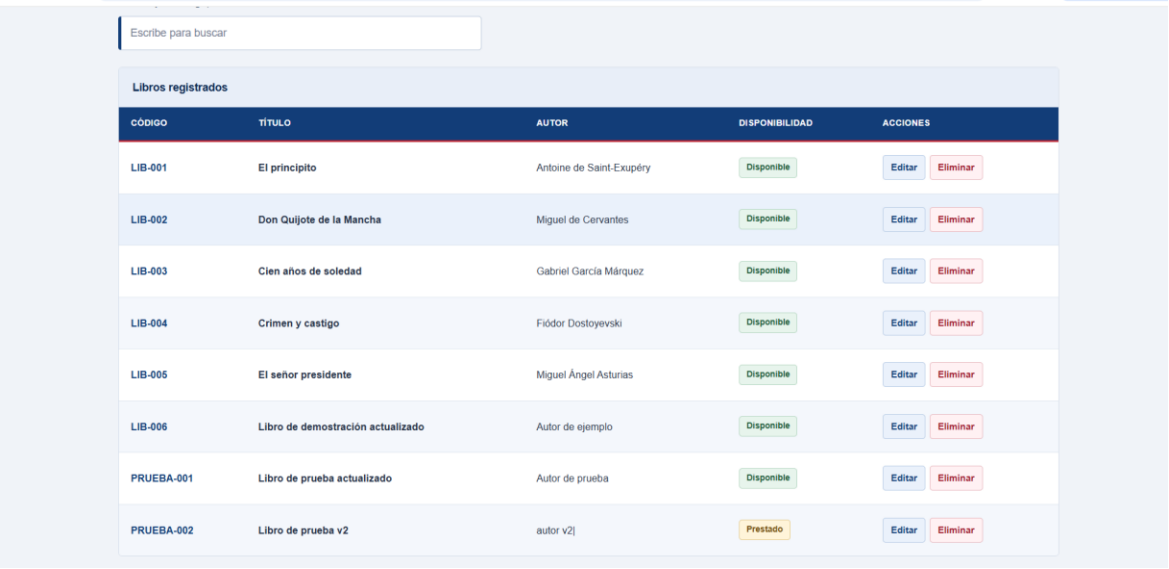


1 Vista de libros

La sección Libros presenta el catálogo de ejemplares y su disponibilidad. Incluye un formulario con los campos código, título, autor e ISBN opcional.

El formulario permite registrar un ejemplar y editar sus datos. La tabla incluye botones para editar y eliminar, mientras que el buscador filtra los registros por código, título o autor.

Antes de eliminar un ejemplar, la interfaz solicita confirmación. Si el libro tiene historial de préstamos, la API rechaza la operación y la página muestra el motivo.

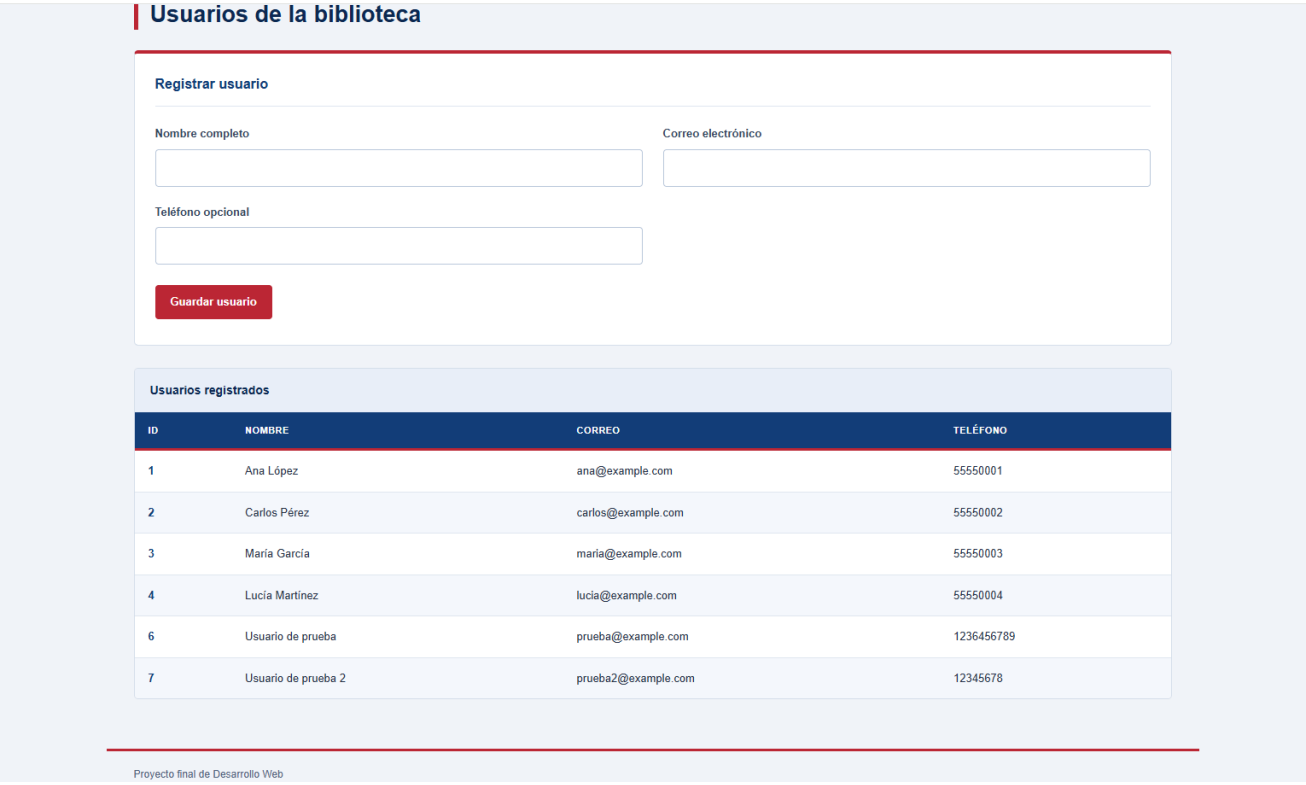


2 Vista de usuarios

La sección Usuarios permite registrar lectores mediante un formulario con nombre, correo electrónico y teléfono opcional. Debajo del formulario se muestra una tabla con los usuarios registrados.

El navegador comprueba los campos obligatorios y el formato del correo. JavaScript valida el nombre y el formato del teléfono, y el backend repite las validaciones antes de guardar los datos.

Cuando el registro es correcto, el formulario se limpia y la lista se actualiza. Si el correo ya existe, se muestra el mensaje de conflicto recibido de la API.



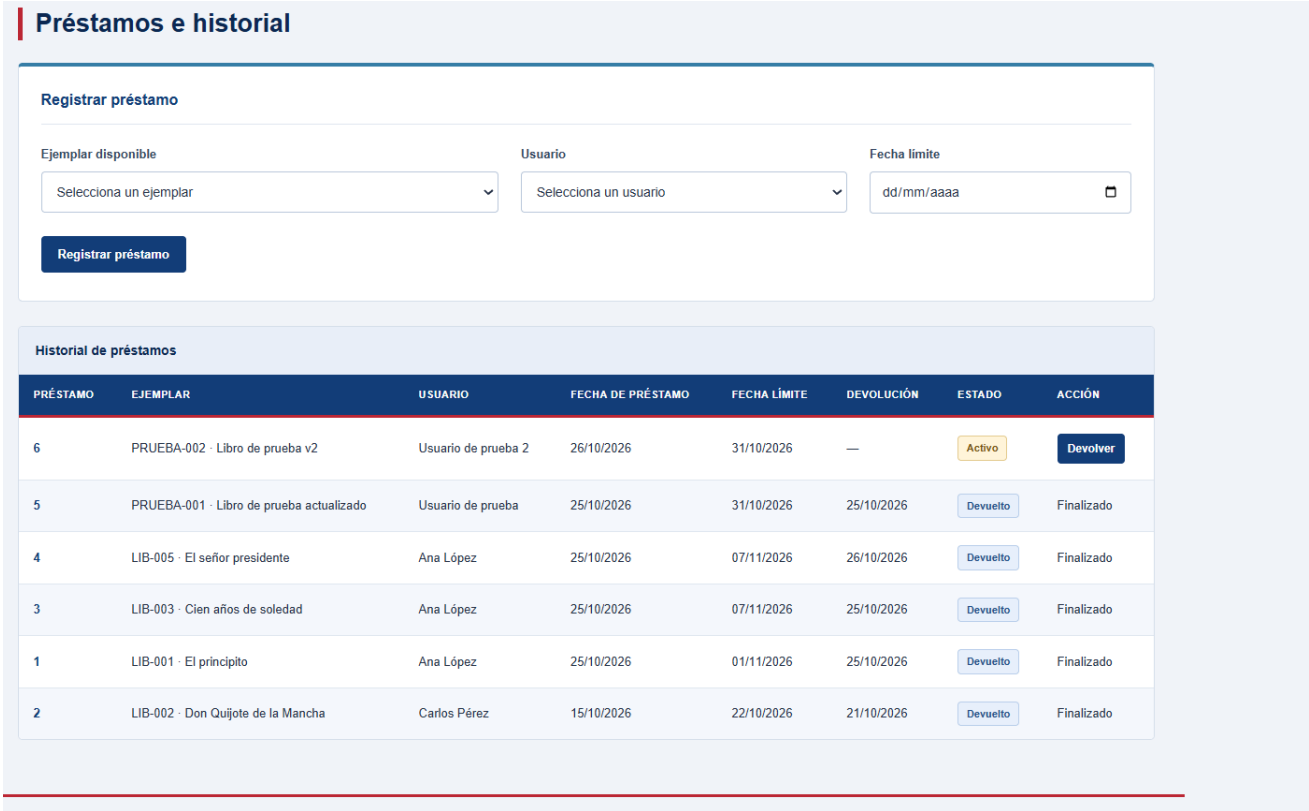
Nota. Elaboración propia. Los registros corresponden a lectores de la biblioteca y no a cuentas de inicio de sesión.

3 Vista de préstamos

La sección Préstamos contiene un formulario para seleccionar un ejemplar disponible, un usuario y una fecha límite de devolución.

El selector de libros muestra únicamente los ejemplares disponibles según la información consultada en la API. El backend vuelve a comprobar la disponibilidad antes de registrar el préstamo, porque el estado podría haber cambiado después de cargar la página.

El historial presenta el identificador del préstamo, el ejemplar, el usuario, las fechas y el estado. Los préstamos pendientes incluyen un botón para registrar la devolución. Los devueltos permanecen visibles como parte del historial.



Nota. Elaboración propia. La devolución actualiza la disponibilidad del ejemplar y conserva su historial.

4 Eventos y comunicación con la API

JavaScript utiliza eventos submit para procesar los formularios, click para los botones e input para actualizar la búsqueda de libros.

En los formularios se utiliza preventDefault() para evitar la recarga automática de la página. Después, se obtienen los valores ingresados y se envían a la API mediante fetch().

La función de comunicación utiliza async/await para esperar la respuesta del servidor. Cuando la solicitud contiene datos, estos se convierten a JSON mediante JSON.stringify() y se envían con el encabezado Content-Type: application/json.

La interfaz comprueba response.ok para identificar respuestas HTTP de error. Si la operación es exitosa, consulta nuevamente los libros, usuarios y préstamos para actualizar las tablas y los selectores.

5 Validaciones y tratamiento de errores

Los formularios utilizan atributos como required, maxlength y type="email". Estas validaciones se complementan con JavaScript y con las comprobaciones del backend.

Durante una operación, el botón utilizado se deshabilita temporalmente para reducir envíos repetidos. Los mensajes informan si la operación terminó correctamente o si ocurrió un error.

Los bloques try/catch permiten controlar errores de las solicitudes. Si el cambio se guarda, pero falla la actualización posterior de la vista, se informa al usuario para que recargue la página sin repetir innecesariamente la operación.

Los datos se incorporan al documento mediante textContent. Los arreglos utilizados en el navegador sirven para presentar y filtrar información; el almacenamiento permanente corresponde a MySQL.

Instalación y ejecución

La instalación requiere Node.js con npm, MySQL Server y un cliente para ejecutar el script SQL. También se utiliza Git para descargar el repositorio.

1 Obtención del código

Desde una terminal se ejecuta:

git clone https://github.com/ximenaguilar/sistema-biblioteca.git

cd sistema-biblioteca

También puede descargarse el repositorio como archivo ZIP y abrirse la carpeta que contiene package.json.

2 Instalación de dependencias

Dentro de la carpeta del proyecto se ejecuta:

npm.cmd install

En la verificación desde una copia limpia se utilizó:

npm.cmd ci

Este último comando instala las dependencias a partir de package-lock.json y requiere que dicho archivo coincida con package.json.

3 Creación de la base de datos

En MySQL Workbench se abre una conexión con una cuenta administradora y se ejecuta database.sql.

El script crea la base biblioteca, sus tres tablas relacionadas y los datos iniciales. Debe ejecutarse sobre una instalación donde no existan previamente las tablas del proyecto.

La carga inicial contiene seis ejemplares, tres usuarios y dos préstamos. No se debe repetir el script completo sobre la base utilizada durante las pruebas.

4 Configuración del usuario de aplicación

Con una cuenta administradora se ejecutan las siguientes instrucciones, sustituyendo la contraseña de ejemplo por una contraseña local:

```
CREATE USER 'biblioteca_app'@'localhost'
IDENTIFIED BY 'REEMPLAZAR_CON_CLAVE_LOCAL';
GRANT SELECT, INSERT, UPDATE, DELETE
ON biblioteca.*
TO 'biblioteca_app'@'localhost';
```

Si el usuario ya existe, se utiliza su configuración existente. La cuenta de la aplicación no requiere permisos para crear o eliminar tablas.

5 Variables de entorno

Se copia el archivo de ejemplo:

```
Copy-Item .env.example .env
```

Después se completan las variables locales:

```
PORT=3000
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=biblioteca_app
DB_PASSWORD="rootsql123"
DB_NAME=biblioteca
```

El archivo .env está excluido del repositorio. La contraseña real no debe incluirse en la documentación ni en las evidencias.

6 Ejecución

Para iniciar el servidor se ejecuta:

```
npm.cmd start
```

La aplicación se abre en:

```
http://localhost:3000
```

La conexión con MySQL se comprueba en:

```
http://localhost:3000/api/health
```

La respuesta esperada es:

```
{
  "estado": "ok",
  "base_de_datos": "conectada"
}
```

La terminal debe permanecer abierta durante el uso de la aplicación. Para detener el servidor se presiona Ctrl + C.

7 Verificación de instalación limpia

Se descargó una copia del repositorio en una carpeta independiente y se instalaron sus dependencias mediante npm ci.

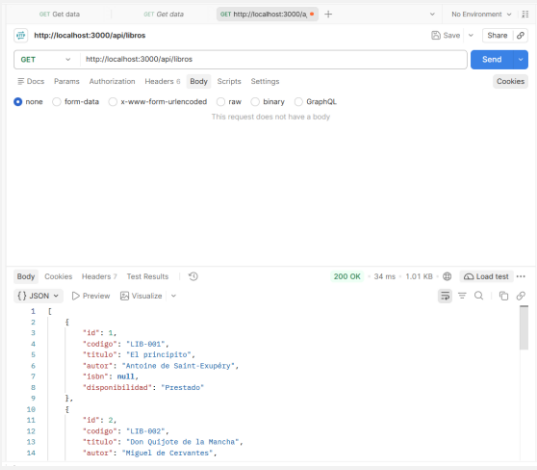
Para comprobar el script sin alterar la base original, se ejecutó una copia de database.sql cambiando el nombre de la base a biblioteca_verificacion. Se verificó la creación de las tablas y la carga de seis libros, tres usuarios y dos préstamos.

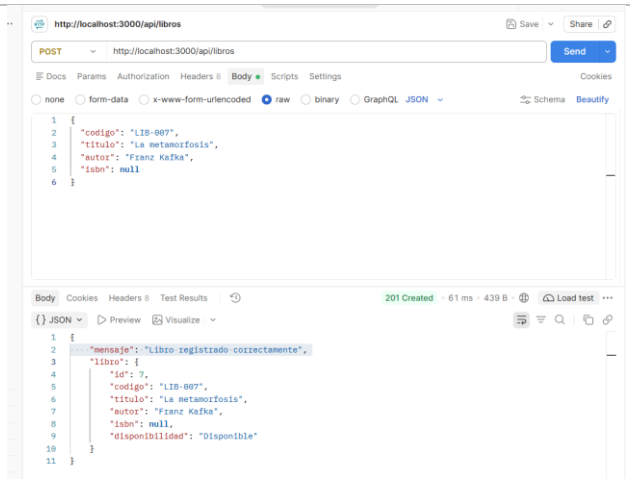
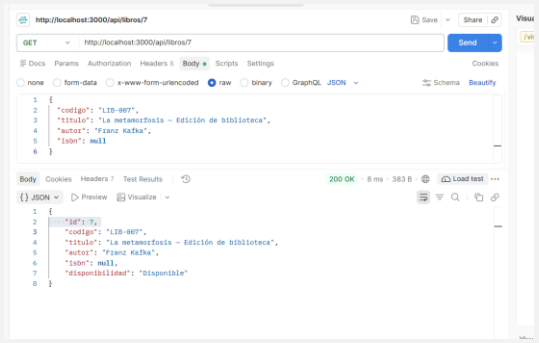
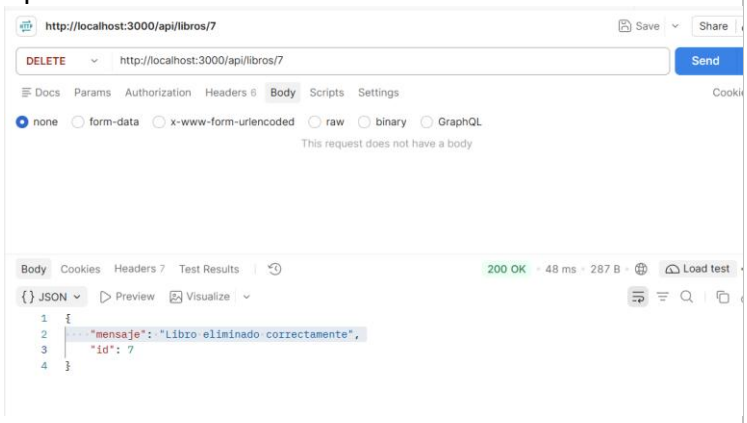
Después se configuró la copia de la aplicación para conectarse con esa base y se comprobó la respuesta correcta de /api/health y la visualización de los datos.

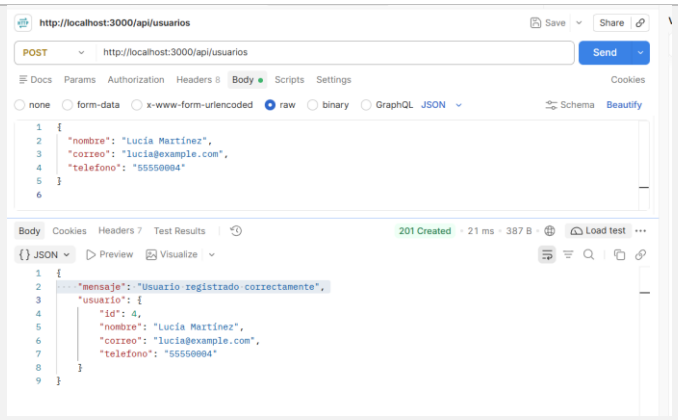
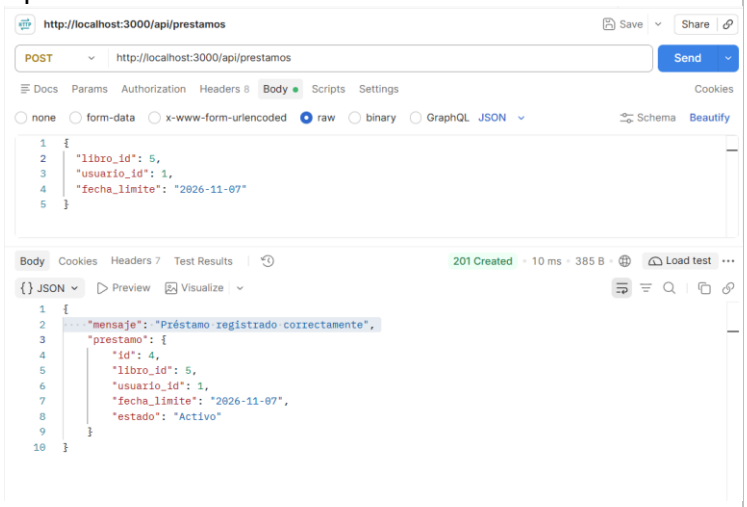
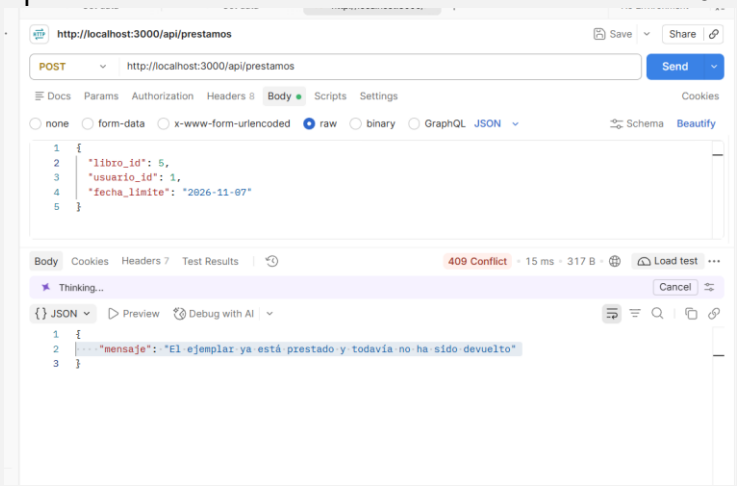
Pruebas y evidencias

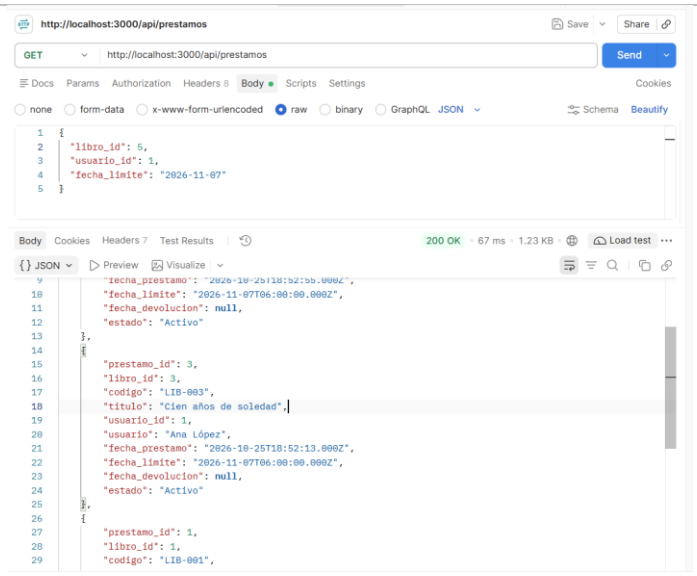
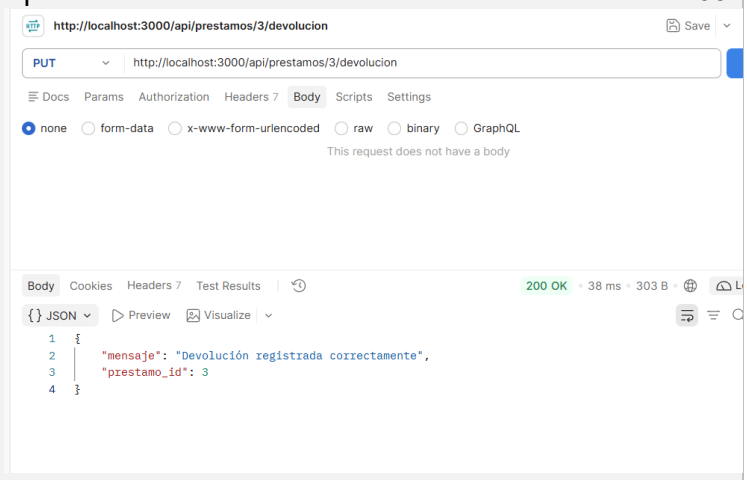
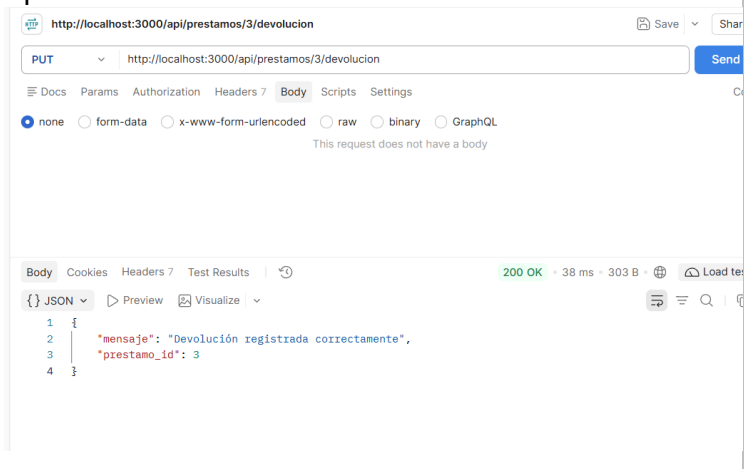
Las pruebas se realizaron mediante Postman, el navegador y consultas en MySQL Workbench. Se comprobaron operaciones válidas, validaciones, conflictos de negocio y conservación de datos.

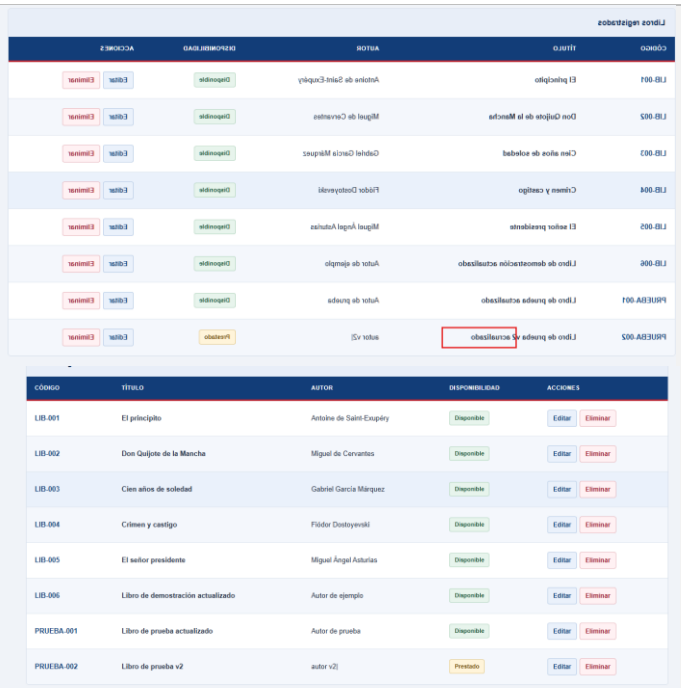
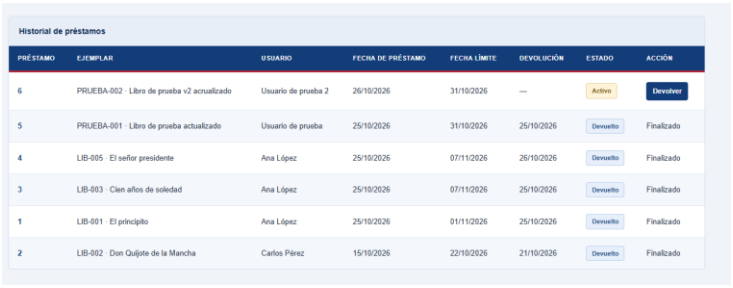
La colección de Postman permite reutilizar las solicitudes. Las capturas documentan los resultados obtenidos durante la ejecución.

Prueba	Esperado	Obtenido	Estado y evidencia
P01 Consulta de libros	HTTP 200 y catálogo con disponibilidad.	Se recibió el catálogo y el estado de los ejemplares.	Aprobado. E01 
P02 Registro de libro	HTTP 201 y nuevo ejemplar.	El ejemplar se registró y pudo consultarse.	Aprobado. E02

			
P03 Actualización de libro	HTTP 200 y datos modificados.	El título actualizado se conservó al consultar.	Aprobado. E03 
P04 Eliminación sin historial	HTTP 200 y consulta posterior 404.	El ejemplar de prueba fue eliminado.	Aprobado. E04 
P05 Registro de usuario	HTTP 201 y usuario visible.	El usuario se registró y apareció en la lista.	Aprobado. E05

			
P06 Registro de préstamo	HTTP 201 y ejemplar prestado.	El préstamo se registró y cambió la disponibilidad.	Aprobado. E06 
P07 Préstamo duplicado consecutivo	HTTP 409 sin crear otro préstamo activo.	El segundo préstamo fue rechazado.	Aprobado. E07 
P08 Consulta de historial	HTTP 200 con ejemplar, usuario y fechas.	Se consultaron préstamos pendientes y devueltos.	Aprobado. E08

			
P09 Devolución	HTTP 200 y ejemplar disponible.	Se registró la devolución y se conservó el historial.	Aprobado. E09 
P10 Persistencia	Conservar datos después de reiniciar Node.js.	Los registros permanecieron disponibles.	Aprobado. E10 
P11 Operaciones desde la interfaz	Formularios y botones conectados con la API.	Se completaron registros, edición y eliminación	Aprobado. E11 y E12

		permitida.	
P12 Préstamos desde la interfaz	Registrar y devolver desde la página.	El historial y la disponibilidad se actualizaron.	Aprobado. E13 

2 Evidencia de persistencia

La persistencia se comprobó deteniendo el servidor Node.js y ejecutándolo nuevamente. Los registros continuaron disponibles, debido a que se almacenan en MySQL y no únicamente en la memoria del backend.

Figura **15**

Consulta de datos después de reiniciar el servidor



Nota. Elaboración propia. La consulta posterior al reinicio conservó los registros.

Dificultades y aprendizajes

1 Dificultades encontradas

Dificultad	Causa identificada	Solución y comprobación
No existía conexión con MySQL	Se contaba con Workbench, pero faltaba instalar y configurar MySQL Server.	Se instaló el servidor y se comprobó la conexión con una consulta SQL.
Organización incorrecta de carpetas	Las carpetas se crearon anidadas en lugar de estar al mismo nivel.	Se reorganizó el proyecto y se comprobó su estructura desde PowerShell.
La interfaz aparecía sin estilos	El nombre del archivo CSS estaba mal escrito.	Se corrigió el nombre y se verificó la presentación en el navegador.
La carpeta docs no aparecía en GitHub	Faltaba contenido versionado dentro de la carpeta.	Se agregó contenido y se publicó mediante Git.
No estaban disponibles las solicitudes en el historial de Postman	El historial consultado estaba vacío.	Se reconstruyeron las solicitudes y se guardaron en una colección exportable.

2 Aprendizajes obtenidos

El desarrollo permitió comprender el recorrido de una operación Full Stack: un evento en el navegador genera una solicitud HTTP, Express valida los datos, MySQL ejecuta la operación y la interfaz presenta la respuesta.

También se distinguió la función de MySQL Server, encargado de almacenar los datos, de MySQL Workbench, utilizado como herramienta de administración.

Las pruebas mostraron la importancia de validar en el frontend y en el backend. Aunque un formulario limite las entradas, la API debe comprobarlas nuevamente porque puede recibir solicitudes desde otras herramientas.

Se comprendió la diferencia entre conservar datos en memoria y almacenarlos permanentemente en una base de datos. Asimismo, se implementaron transacciones y bloqueos para coordinar el registro de préstamos de un mismo ejemplar.

El uso de Git y GitHub permitió mantener avances identificables, mientras que la instalación limpia ayudó a comprobar que el proyecto podía ejecutarse a partir de los archivos publicados.

Conclusiones y mejoras

1 Conclusiones

1. Se implementó una aplicación web que integra HTML, CSS y JavaScript con una API REST desarrollada en Node.js y Express y una base de datos MySQL.
2. Las pruebas realizadas comprobaron el CRUD de libros, el registro y consulta de usuarios, los préstamos, las devoluciones y la consulta del historial.
3. El sistema identifica la disponibilidad de los ejemplares y rechaza el registro consecutivo de un segundo préstamo pendiente para un ejemplar ocupado. El mecanismo de transacciones y bloqueo está implementado; su prueba con solicitudes simultáneas permanece pendiente.
4. La información se conservó después de reiniciar Node.js. Además, se comprobó la creación de una base independiente mediante el script SQL y la ejecución desde una copia del repositorio.
5. Las consultas parametrizadas, las variables de entorno y el usuario MySQL con permisos limitados incorporan las medidas básicas de seguridad previstas para el alcance académico local.

2 Mejoras propuestas

Como trabajo futuro se propone incorporar autenticación y roles para controlar quién puede ejecutar las operaciones del sistema.

También pueden añadirse filtros al historial, reservas de ejemplares, reportes y recordatorios de devolución. Estas funciones requieren definir reglas adicionales y se mantienen fuera de la implementación actual.

Antes de ampliar el acceso a otros equipos, se deben completar las pruebas pendientes, revisar la configuración de producción y establecer mecanismos de autenticación y autorización.

Referencias y anexos

Anexos y ubicación de entregables

Entregable	Ubicación
Repositorio de código	https://github.com/ximenaguilar/sistema-biblioteca
Aplicación local	http://localhost:3000
Script de estructura y datos iniciales	database.sql
Instrucciones de instalación y uso	README.md
Colección de Postman	docs/Biblioteca.postman_collection.json
Capturas de pruebas	Carpeta docs
Informe final	Informe_Final_Biblioteca.docx
Guion de exposición	[Agregar ubicación cuando esté preparado]
Fecha de última verificación	[Fecha real de la última prueba]

La dirección http://localhost:3000 corresponde a la aplicación ejecutándose en la computadora local. Para utilizarla en otro equipo, primero deben seguirse las instrucciones de instalación del README.